

Computer Systems Performance Exam

Mie Jonassen, Michal Rynowiecki

July 29, 2026

1 Project 1: Reproducing Data Partitioning Performance Experiments on a Modern Multi-Core Processor

1.1 Introduction

With the rapid growth of big data across many different sectors, efficient partitioning is becoming more and more relevant. Using multiple cores to partition data may provide more efficient processing times but also creates other challenges related to coordination of work and accesses to shared resources between multiple active workers.

Knowing how to efficiently exploit parallelization of workloads and trade-offs is important for current and future data-intensive applications relying on sorting and dividing of data **without extra overhead**. Parallelizing workloads does not increase the throughput linearly in the number of workers, as workers are required to coordinate with all other workers accessing the same resources. The overhead only pays off for larger workloads and an efficient strategy for this coordination work makes a big difference in the overall efficiency **as coordination is the main holdback**.

In this project we investigate 2 different methods for data partitioning with the goal of analyzing the effect on data throughput when varying the number of threads and partitions for a simulated workload. We also aim to investigate how the change in hardware impacts the results of the experiments over time, by relating our findings to those of previous work. ¹

1.2 Methodology

The goal of the project is to investigate data partitioning methods' behavior when running on multicore hardware. The project aims to reproduce selected experiments from [CR08], investigating how results compare on different hardware.

1.2.1 Experimental Design

The system we are investigating consists of two selected partitioning methods at the application level (Concurrent Output & Parallel Buffers, as described in [CR08] and elaborated in 1.2.2) running on a multicore CPU server with a general-purpose Linux OS. The architecture of the server, referred to as Dionysos, can be seen in Figure 1, displaying the memory hierarchy and cores. Additional specifications of Dionysos can be seen in Table 1.

We are measuring throughput as the main metric for comparing systems by examining the number of tuples processed per second. In practice, we do this by measuring the total data partitioning

¹The code for this report can be found on GitHub at https://github.com/Mie-Jonasson/computer_systems_performance.

CPU Model	Intel(R) Xeon(R) Silver 4109T
Clock	2 GHz
OS	Ubuntu 24.04, Linux kernel 6.17.0-23-generic
Page Size	4KB (default)
Compiler	g++ -std=c++17 -O2 -pthread

Table 1: Additional Specs for Dionysos; memory hierarchy and core layout are shown in Figure 1

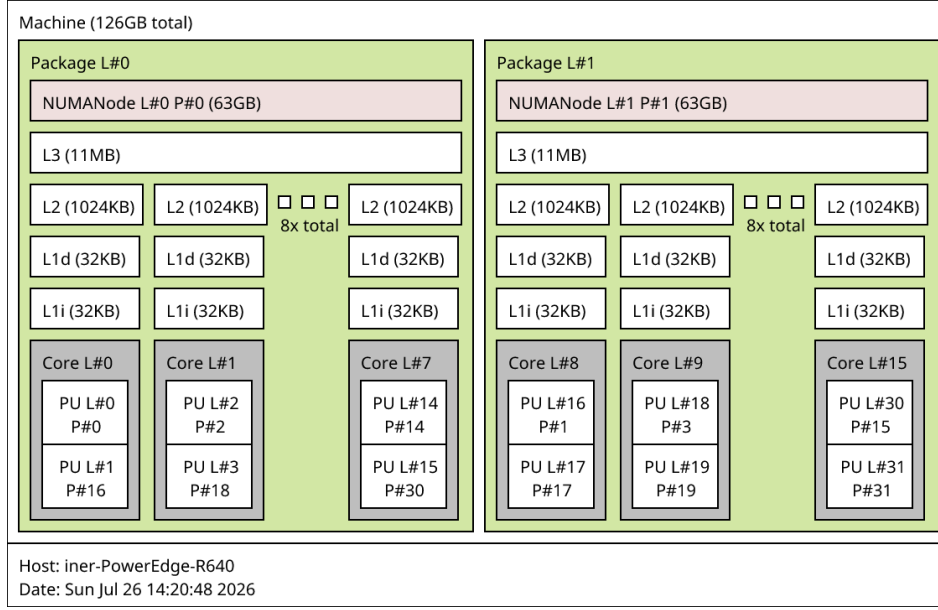


Figure 1: *Visual Representation of the memory hierarchy and compute of the Dionysos Server used for Experiments*

time, starting the timer after threads and data has been initialized and stopping it once all data has been successfully partitioned. This metric gives us comparable insights into how efficient each of the partitioning methods is for processing large batches of data. We use the commandline-tool 'perf' to obtain additional hardware-level metrics to provide insights into what is happening under the hood and enrich the analysis of our results.

When comparing the two systems, we run experiments across a selected number of hash bits and threads. The levels for each parameter can be seen in Table 2. Hash bits describe how many bits we use to decide which partition the item should land in, with each bit having a value of 0 or 1. The hash bits therefore describe an exponentially growing function of number of partitions, denoted by: $\# \text{ Partitions} = 2^b$. The number of threads are also varied and describe how many threads are taking part in the data partitioning task. Each thread always processes an equal amount of the workload provided which is divided as contiguous slices by the main thread before starting work.

Parameter	Levels
Hash Bits	[1:18]
# Threads	[1, 2, 4, 8, 16, 32]
Core Affinity	[w. affinity, wo. affinity]

Table 2: *Levels for each parameter in the experimental setup*

Our experiments rely mainly on quantitative measurements logged on the software-level of the system, with analytical insights added as relevant. We run the experiments across all combinations of parameters and systems for 3 trials each. Due to **runtime concerns on runs with a large number of partitions**, we did **not find it feasible to run more trials than this**.

Our workload consists of uniformly generated tuples of 8-byte keys and 8-byte payloads, matching the setup of the paper we are reproducing. This allows us to mimic the experiment more closely and have comparable setups. We choose an arbitrary size for the workload of 10.000.000 tuples, which is sufficiently big to gain insights while remaining feasible to run in our time frame.

1.2.2 Partitioning Methods

For this project we are investigating two of the four data partitioning methods from the paper [CR08], namely 'Concurrent Output' and 'Parallel Buffers'. What these two methods have in common is

the way the buffer works. It is initialized as a shared data structure across threads with a counter associated with each bucket, indicating the current head of the stack. When a thread wants to insert an element, it should increase this counter with atomic operations or a lock. **In our implementation we use a mutex lock over the index tracking data structure.**

The main difference between the two partitioning approaches is the approach to locking. The 'Concurrent Output' method uses the classical approach to multi-threading of obtaining the lock on the list of bucket head indices on every write whilst 'Parallel Buffers' calculates a batch of bucket indices before obtaining the lock and updating the indices for the entire batch. The main idea of the 'Parallel Buffers' is to process more data independently such that there is **less overhead of obtaining and releasing the lock for the shared data structure.** I.e. we are comparing a classic ad-hoc lock-then-write to a batched approach.

We simplify the approach to batching data for Parallel Buffers and keep it **fixed and 100 tuples** across all runs. This differs from [CR08], where **batch sizes are scaled down as hash bits increase** as the contention problem we are trying to resolve becomes less prominent because threads are distributed across larger ranges.

1.2.3 Implementation Details

When implementing the experiments we needed 2 shared resources for the buffers; the buffers themselves and a separate data structure to keep track of the current index of the head of a given buffer. The buffers are initialized as an array of length 2^b , with b being the number of hash bits. The tracking of indices is managed in another array containing integers indicating the next available index in a given buffer. When reading and writing to this index-tracker a lock must be obtained by the thread **to ensure sequential consistency and avoiding two threads writing on the same index of a buffer.**

The use of a mutex lock on the entire array may hinder performance as opposed to an approach using an array of atomic integers. **We chose to use a mutex lock for simplicity of the implementation even though it increases the likelihood and severity of contention between threads.**

We implemented everything in C++17 using `std::thread` and a shared `std::mutex` for multithreading and protecting the per-bucket write indices. **Input data is generated once per process** as uniformly random 8-byte keys with a constant 8-byte payload (arbitrarily set in code as a constant value), then split into contiguous, equal-sized chunks for each thread. **Before partitioning, threads synchronize on a barrier** so the timed interval excludes data generation, buffer allocation, and thread creation. **The buffers are initialized as empty arrays of double the expected number of items in the bucket** (since we are working with uniformly distributed data, we assume each bucket will contain equally sized chunks). If a buffer overflows, it is resized with the same lock as the index-tracking data structure. We went with this approach in order to have a reasonable assumption that we would not need to resize while partitioning without having a too large data storage overhead to initialize.

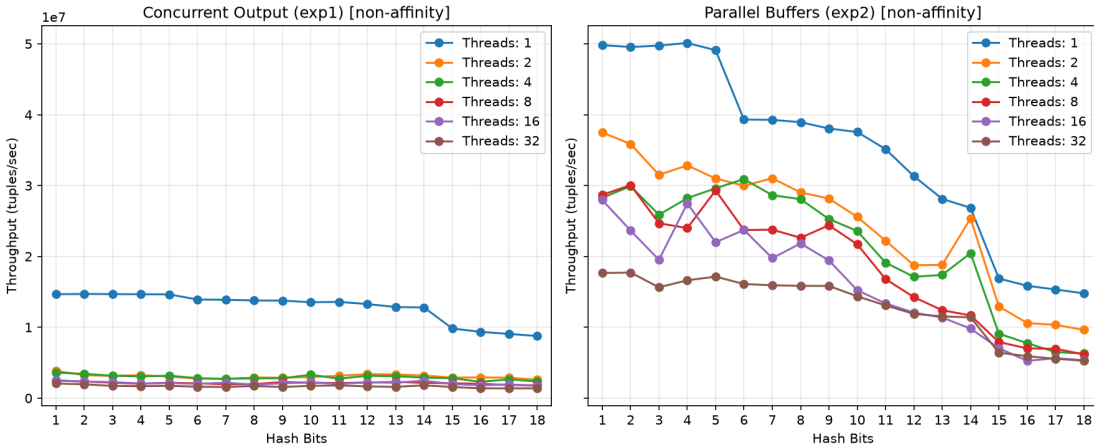


Figure 2: Overall throughput (tuples / sec) of two chosen algorithms for different number of hash bits and threads with a workload of 10 million tuples

We run the experiments both with and without core affinity set, to examine how fixing threads to a specific core impacts the performance of each of the chosen partitioning methods. We ensure threads are evenly distributed across the 16 cores of Dionysos by assigning them according to $t_i \% C$, with t_i being the thread index and C being the total number of available cores. Note, as can be seen in Figure 1, that **Dionysos has 32 Hardware Threads** distributed among the 16 cores.

1.3 Results

We initially ran the experiments without core affinity, to investigate the default behavior when not directly managing thread assignment on cores. The results obtained from this run can be seen in Figure 2. We observe a trend of somewhat stable throughput up until 14 bits where 'Parallel Buffers' then drops. We deem that this is likely related to **"hitting a wall"** due to an **increased number of writing locations along with the increased size of the hash key itself** when batching calculations, as the size doubles for every hash bit. This observation is further discussed in Section 1.3.2.

We see that the 'Concurrent Output' achieves better performance with a single thread, and generally only gets lower throughput when increasing the number of threads. The 'Parallel Buffers' algorithm generally has higher throughput across thread counts compared to 'Concurrent Output', but still show a similar trend of lower throughput as thread count increases. The 'Parallel buffers' also has a larger range of throughputs across hash bits, indicating that the method is impacted more by the number of partitions. **We do note that using atomic integers over a lock on the entire array of indices might have produced higher throughput for both methods and might have provided a different result.**

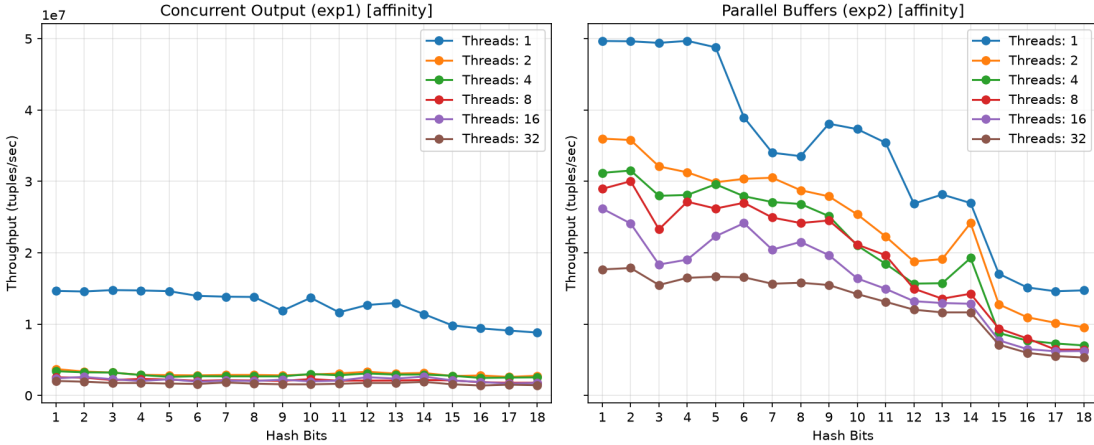


Figure 3: Overall throughput (tuples / sec) of two chosen algorithms for different number of hash bits and threads with a workload of 10 million tuples using Core Affinity.

The results when running with core affinity on the threads show a similar story, as seen in Figure 3. The relative ranking of the methods remains the same, indicating that the reduced lock contention of the 'Parallel Buffers' approach succeeds in making the partitions run faster, while introducing core affinity makes little difference on the results. We further discuss the hardware level performance insights in below sections.

1.3.1 Hardware Level Performance Insights

We collected performance statistics about the overall runs by running 'perf' with 3 trials for each experiment combination. The tool give us insights into the hardware-level behavior impacting the runtime. We **analyze the trends around instructions per cycle (IPC) and cache-miss counts, averaged across hash bits for each thread count**, which is available in Table 3.

For 'Concurrent Outputs', we see a sharp increase in cache misses and decrease in IPC, indicating that **cores quickly end up spending a lot of time waiting rather than performing useful work**. We hypothesize that **this is because of the heavy reliance on the single mutex**. This fits with our expectations and analysis of the **single mutex being a bottle neck** for the 'Concurrent Outputs' method when

Method	Thread-Count	IPC	Cache-Misses
Concurrent Output	1	1.02	$0.71 * 10^7$
Concurrent Output	2	0.38	$4.65 * 10^7$
Concurrent Output	4	0.31	$7.55 * 10^7$
Concurrent Output	8	0.24	$13.70 * 10^7$
Concurrent Output	16	0.14	$17.00 * 10^7$
Concurrent Output	32	0.08	$20.00 * 10^7$
Parallel Buffers	1	1.19	$0.70 * 10^7$
Parallel Buffers	2	0.96	$1.44 * 10^7$
Parallel Buffers	4	0.91	$1.59 * 10^7$
Parallel Buffers	8	0.88	$1.69 * 10^7$
Parallel Buffers	16	0.90	$1.67 * 10^7$
Parallel Buffers	32	0.86	$1.65 * 10^7$

Table 3: Average IPC & cache-miss counts across hash bits for runs with core affinity

adding more threads, as the coordination at each write becomes too heavy a task compared to the partitioning itself.

'Parallel Buffers' does not seem to have the same struggles with thread contention, as IPC only drops slightly and both IPC and cache-misses remain rather stable from 4 threads and up. The batching of calculations before obtaining the lock reduces the overhead of needing to await the lock by having less transactions. The stable nature indicates that the additional threads are not hindering the work of each other, but also that they do not have improved efficiency.

Both methods lose throughput as the number of partitions grow, which differs from the results of [CR08]. This may likely be caused by the choice of locking mechanism over atomic integers in an array. A future iteration may investigate this different approach to ensuring mutual exclusion of the buffer indices.

1.3.2 Throughput Drop

In an attempt to explain the throughput drop-off after 14 hash bits, we plot the throughput against the cache-misses for the single thread experiments in Figure 4, with a marker line placed at 14 hash bits. The figure clearly shows a sharp increase in cache misses after 14 hash bits across methods and core affinity. Looking at the single-thread experiment is particularly interesting for this trend, as we do not have thread contention and therefore have less factors impacting the throughput.

The throughput drops and stays rather stable at a lower volume for 15-18 hash bits. This is expected from the memory hierarchy when the working set becomes too large to fit in the cache memory closer to the core. When the working set no longer fits in the working memory, data transfers to lower level caches are needed. With a low page size of only 4KB (compared to the 256MB page size in [CR08]) this becomes even more challenging for large volumes of data, as a lot of data transfers have to be performed to move the data between different caches.

These observations match the findings of [CR08], stating that partitioning performance degrades when output cardinality exceeds on-chip resources. The bottleneck at higher number of hash bits is shared among the two methods and is therefore an algorithm-agnostic bottleneck not directly related to the same thread contention issues as previous observations.

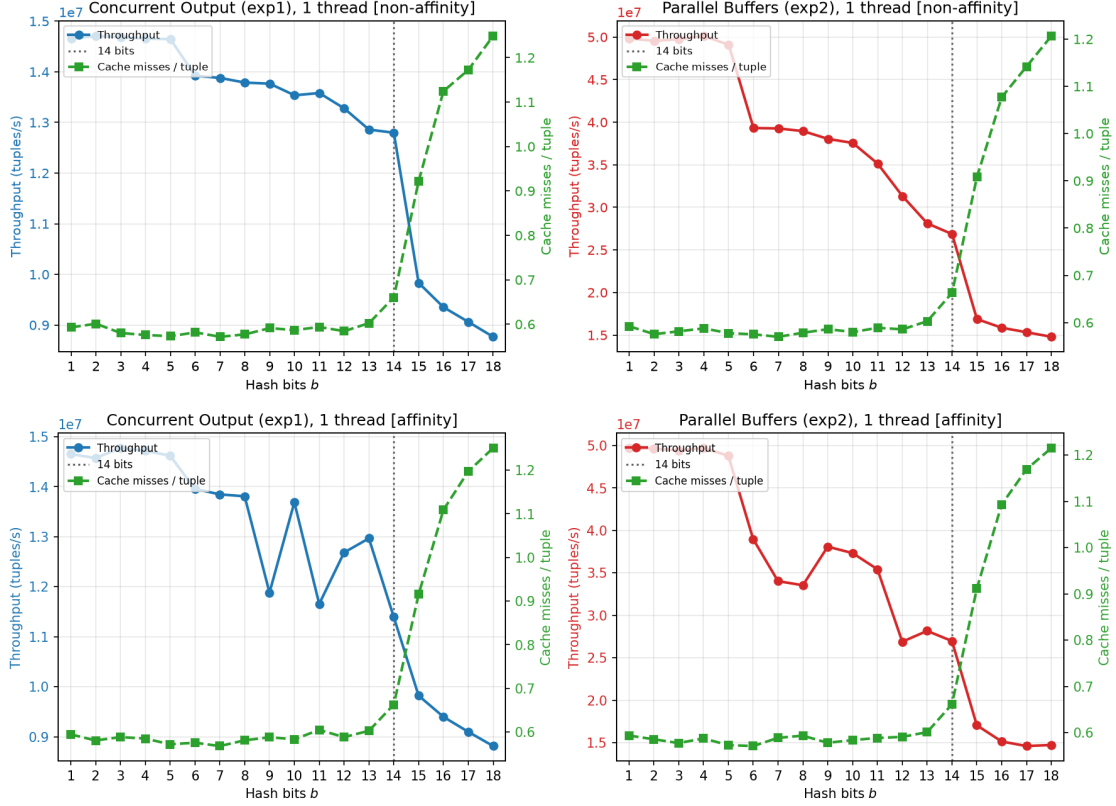


Figure 4: *Throughput and cache misses plotted for 1 thread on each of the methods and affinities. Note, that y-axes are scaled and do not start at 0 to zoom in and get a clearer picture of the development.*

1.4 Conclusion

In this project we compared two partitioning methods from [CR08], Concurrent Output and Parallel Buffers, on Dionysos. We varied the number of threads and hash bits, ran with and without core affinity, and used ‘perf’ to better understand the trends we saw in throughput.

Overall, **Parallel Buffers performs better across experiments**. Because it batches work before taking the lock, there is less contention on the shared index structure. This shows up in the ‘perf’ results as well: IPC stays fairly high and cache misses do not explode when adding threads. Concurrent Output goes the other way. Throughput drops as we add threads, IPC falls sharply, and cache misses rise a lot. That **fits with the method locking on every write, so threads spend more time waiting than doing useful work**.

We also see a drop in throughput after around 14 hash bits for both methods. Looking at the single-thread runs in Figure 4, this lines up with a sharp increase in cache misses. **At that point the number of partitions gets large enough that the working set no longer fits well in cache**, especially with the default 4KB page size. So this bottleneck seems more related to the memory hierarchy than to how the two methods coordinate writes, which matches the general findings in [CR08].

Finally, **setting core affinity did not change the main conclusion**. The ranking between the methods stays the same, and affinity does not remove the lock contention for Concurrent Output or the throughput drop at high hash bits. In short, **how threads coordinate writes matters a lot until the working set grows too large to fit in working memory; after that, cache pressure becomes the main limit for both approaches**.

2 Project 2: Performance Analysis of Computing Hardware for Neural Network Training

2.1 Introduction

Machine learning hardware is increasingly deployed across various domains, particularly within data centers running Multimodal Large Language Models (MLLMs). However, **large-scale compute resources remain inaccessible to most individuals and smaller organizations**. It is therefore essential to evaluate **how accessible hardware options perform when training neural networks**. This study analyzes three commonly available compute platforms: commercial GPUs, CPUs, and Apple Silicon.

To evaluate hardware performance, **we finetune a pre-trained neural network** on an image recognition task. Fine-tuning adjusts existing model weights to a new dataset by evaluating a loss function via forward propagation and computing gradients through backpropagation. **GPUs accelerate this process significantly because these operations rely primarily on large matrix multiplications**, which GPUs execute in parallel across **thousands** of cores, unlike CPUs, which execute tasks sequentially across fewer cores. ²

2.2 Dataset

We use the common dollar dataset [RDK⁺22] which contains 38,479 images collected from 63 different countries, tagged from a set of 289 possible topics. Each image is of resolution 4064x4064 pixels. The dataset’s metadata includes demographic information such as region, country and total household monthly income. **The only part of the dataset metadata utilized in our experiments are the topics**.

2.3 Methodology

To examine the performance differences between different hardware setups, we followed a finetuning procedure for image classification³ to finetune a Vision Transformer (ViT) [WXD⁺20, DDS⁺09] model pre-trained on ImageNet-21k (14 million images, 21,843 classes) at resolution 224x224, and fine-tuned on ImageNet 2012 (1 million images, 1,000 classes) at resolution 224x224. **The model has a total of 86.6M parameters**. We replaced the final classification layer with a **new, randomly initialized classification layer fit with output size 289**, corresponding to the number of topics in the dataset. In the training loop, we scaled each image in the common dollar dataset to size 224x224 and recast the format to RGB. The training configuration used across all experiments can be seen in Table 4. Additional variations from this configuration are described further in below sections.

Table 4: Training configuration used for all experiments.

Parameter	Value
Epochs	4
Batch size	8
Learning rate	0.002
Optimizer	Adam
Python version	3.14.1
PyTorch version	2.10

2.3.1 GPU

We used three GPUs, **V100 with 32GB VRAM, A100 with 40GB VRAM and A30 with 24GB VRAM**. They were **supported by 4 CPUs and 20GB of memory**. Every 5 milliseconds we measured GPU utilization, VRAM utilization, temperature and power draw using **nvidia-smi**. For the GPU we also decided to run experiments with **batches of size 16 and 32** to gain insights into how the batch size would affect the performance during training in this setup.

²git repo: https://github.com/michal-rynowiecki/project_two_sys

³<https://huggingface.co/blog/fine-tune-vit>

2.3.2 CPU

We used two different CPU experiment setups, with 8 and 16 cores respectively. Each run had 48GB of RAM. We measured CPU utilization, number of cores active and memory usage using the cgroup CPU accounting file.

2.3.3 Apple Silicon

For Apple Silicon, we used an M3 Macbook Air with 24 GB RAM and 8 cores (4 Performance and 4 Efficiency), and the MPS library. The accuracy of the Macbook ratings is lower, because they were ran on a laptop which had other unrelated processes active. Despite the best efforts to run the experiments in isolation, it is not possible to achieve the same degree as when using an HPC cluster with SLURM.

2.4 Results

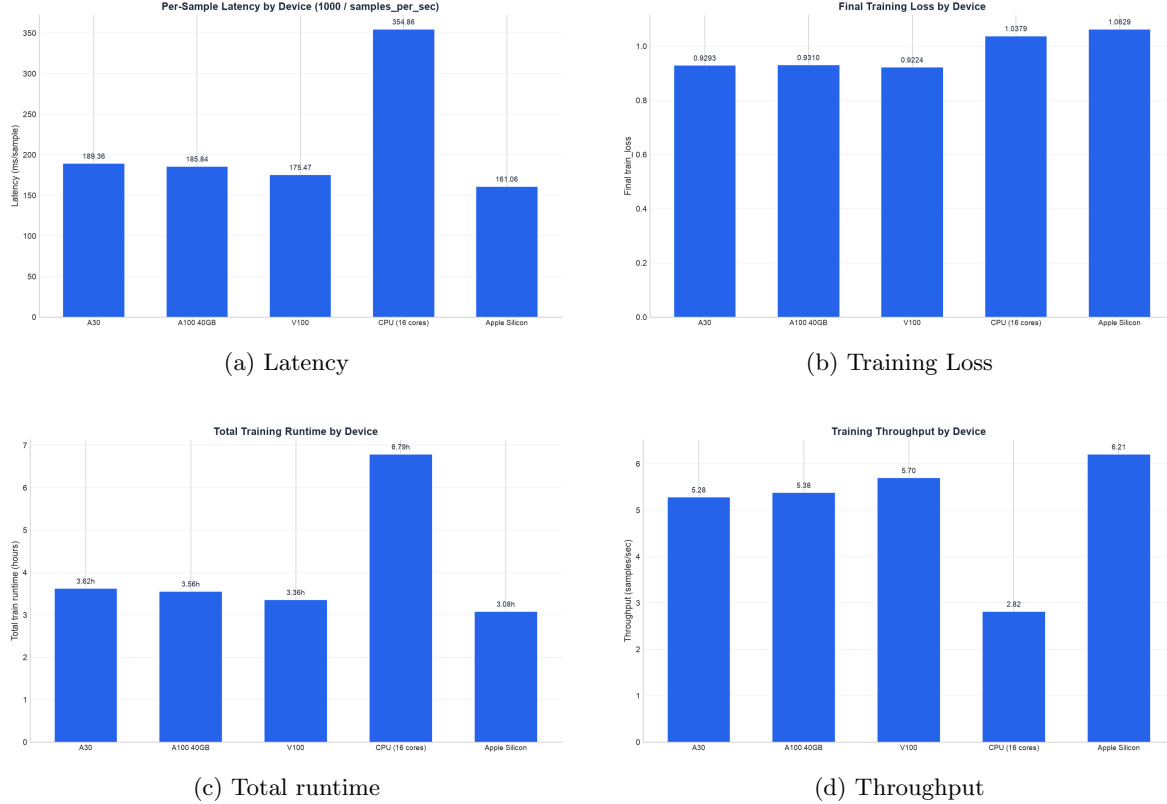


Figure 5: Comparison of the training results for all methods

Figure 5 shows that training throughput is generally comparable across accelerators, hovering between 5 and 6 samples per second. The standalone CPU is a notable outlier, processing only 2.82 samples per second. This performance gap is expected, as both the CUDA and MPS frameworks exploit hardware-level parallelism for matrix multiplication. Consequently, total training times are closely aligned between the cluster GPUs and the MacBook, with the MacBook achieving the fastest overall time (3.08 hours) and the CPU the slowest (6.79 hours). Final training loss is also consistent across all configurations, though slightly lower on the cluster GPUs.

While raw training times are similar, energy expenditure differs significantly. As shown in Figure 6, the cluster GPUs sit idle for approximately 80% of the training cycle, whereas the MacBook GPU, as seen in figure 9, remains consistently active. Because enterprise GPUs draw substantial baseline power even while idle, their total energy consumption is significantly higher than that of the MacBook. This indicates that the MacBook’s competitive runtime is not due to superior raw compute power over production-grade GPUs, but rather to system-level factors. The small batch and network sizes fit easily

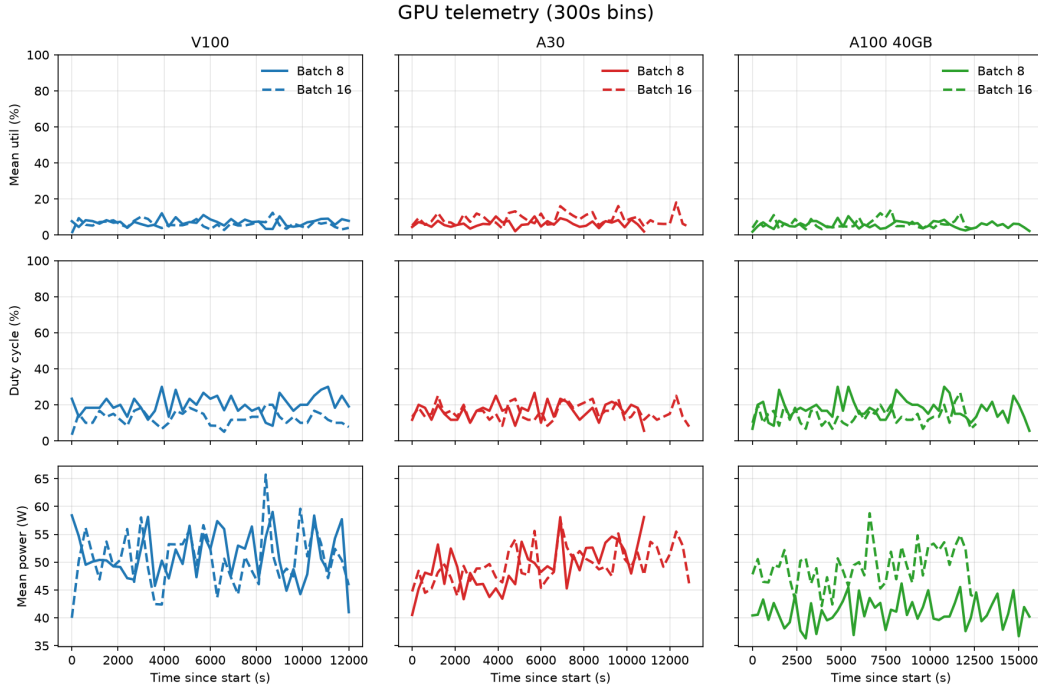


Figure 6: Comparison of metrics for GPUs

within standard memory, neutralizing the high-capacity VRAM advantage of enterprise GPUs. The unified memory architecture of the M3 chip minimizes host-to-device data transfer latency, avoiding the PCIe bus bottlenecks common in SLURM cluster environments. Data loading on the HPC cluster may be constrained by the assigned host CPU, whose specific hardware allocation could not be isolated in the shared cluster environment.

2.4.1 GPUs

In Figure 6 we compare the mean utilization, duty cycle and mean power consumption during the experiments across the three previously stated GPUs for two batch sizes, 8 and 16. Overall, the GPUs show very similar utilization patterns, with the main difference being that the V100 appears to consume more energy than the A30 and A100.

Since the workload operates on image data, the images must first be resized and transformed on the CPU before they can be processed by the GPU. These CPU operations become a bottleneck, causing the GPUs to spend a large portion of the execution waiting for new data. As a result, the GPU duty cycle remains below 50% for most time bins, and the average GPU utilization is also low. This indicates that the GPUs are underutilized. At the same time, the average power consumption remains relatively stable, meaning that energy is still being consumed even while the GPUs are waiting for the CPU. This leads to both inefficient use of GPU resources and unnecessary energy consumption.

Although there is no clear difference in average GPU utilization between the V100, A30, and A100, the V100 shows much greater variation in power consumption and occasionally reaches nearly twice the power draw of the A100. It is also noteworthy that the A100 generally consumes less power than the A30 while exhibiting a similar utilization pattern.

Figure 7 shows training latency and throughput as a function of batch size for the same three GPUs. For the A30, both metrics improve consistently as batch size increases: latency decreases and throughput increases from batch size 8 to 32. The V100 shows a similar but weaker trend, with latency and throughput staying roughly constant between batch sizes 8 and 16 before improving at batch size 32. The A100, in contrast, shows a non-monotonic pattern: latency decreases and throughput increases when moving from batch size 8 to 16, but this trend reverses at batch size 32, where latency increases again and throughput drops below its batch-size-16 value.

This behavior for the A100 is consistent with the CPU-bound nature of the pipeline discussed

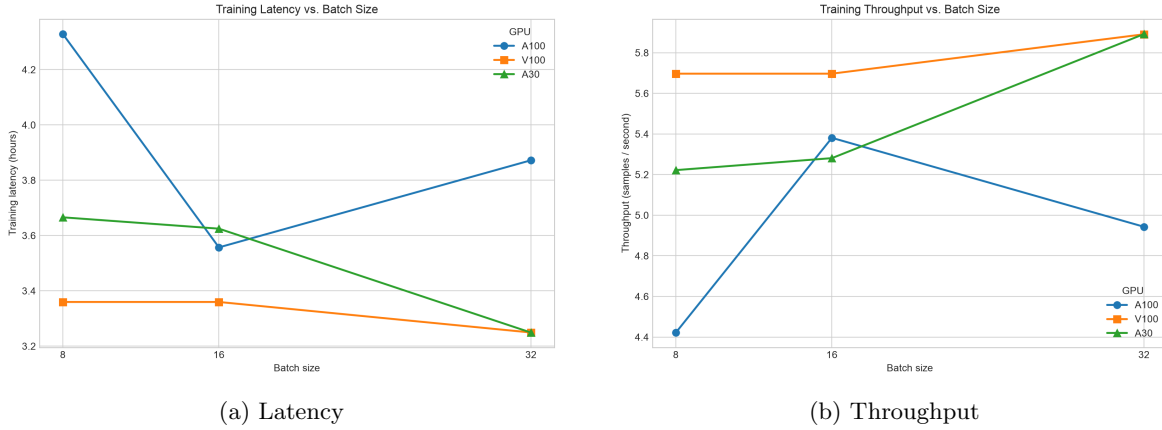


Figure 7: GPU performance based on the batch size

above. Since image resizing and transformation on the CPU limit how quickly the GPU receives new data, **increasing the batch size does not guarantee a proportional reduction in training time**. Once the CPU side pre-processing becomes the bottleneck, further increases in batch size can shift where GPU idle time accumulates rather than reduce it. **This would explain why the A100, which has the highest raw compute capability of the three GPUs, does not benefit from a larger batch size at the same rate as the A30.**

Across all batch sizes, the A30 and V100 achieve lower or comparable latency to the A100 despite their lower compute capability, which further supports the interpretation that the CPU data pipeline, rather than GPU compute, is the limiting factor in this setup.

2.4.2 CPU

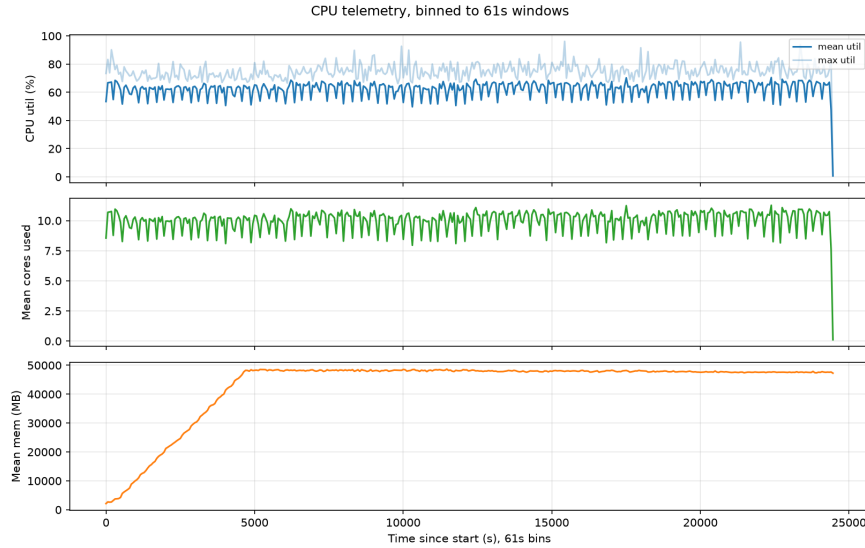


Figure 8: CPU usage metrics

The duty cycle appears to be lower for the higher batch, which supports the hypothesis that the GPUs must wait for the CPUs and also for them to move from the memory. **Since it takes less time for a CPU to process 8 images rather than 16, it makes sense that the GPU would be idle less of the time.**

Figure 6 illustrates the effect of batch size on GPU performance. While increasing the batch size slightly improves throughput by utilizing GPU memory more effectively, the overall performance gains

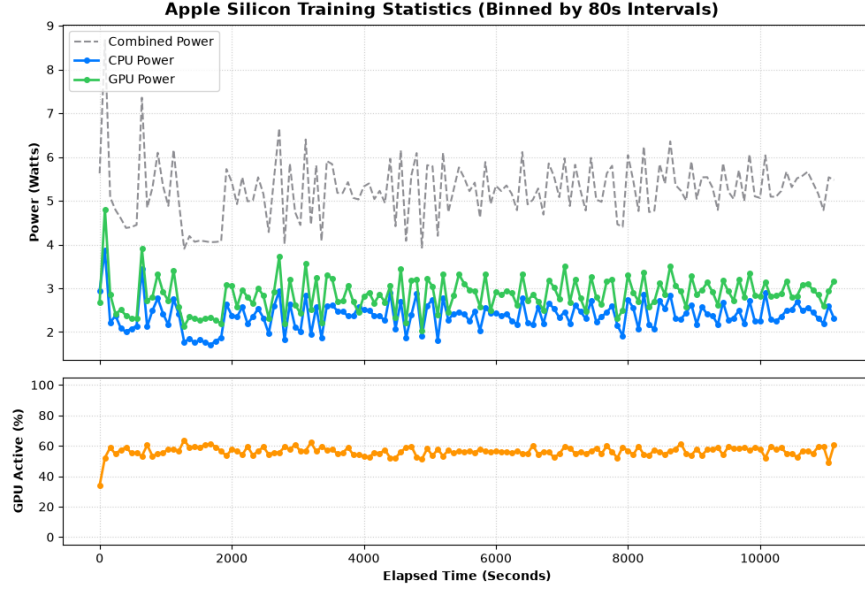


Figure 9: MacBook usage metrics

remain modest. This behavior suggests that the primary bottleneck lies in CPU-side image processing and data transfer to the GPU. Because the dataset is loaded lazily, it cannot be held entirely in GPU memory, meaning larger batch sizes alone cannot fully resolve the throughput limitation.

Figure 8 illustrates resource usage during the CPU training phase, using a 16-core, 40GB RAM configuration. Compared to the GPU trial, system memory usage is significantly higher, quickly consuming almost all available RAM. This heavy memory footprint occurs because system RAM must simultaneously store the model’s parameters, layer activations, gradients, and optimizer states, along with the cached dataset and data-loading buffers.

Processor utilization fluctuates between 7 and 10 active cores, demonstrating solid multi-core engagement without reaching full capacity. This oscillation is primarily driven by memory bandwidth saturation and thread synchronization overhead. When CPU cores process large matrices, they frequently have to pause and wait for data to be read from or written to RAM. [GG08]

Furthermore, while matrix operations are parallelizable, traditional CPUs are created for sequential workloads, rather than the massive data parallelism handled by GPU architectures. Finally, because the CPU handles both data preprocessing and model training in this setup, system resources experience contention. Cores must constantly context-switch between preparing incoming data batches and calculating gradients, preventing a steady 100% utilization across all 16 cores.

2.4.3 Apple Silicon

Figure 9 presents the hardware usage statistics when training on the Apple M3 architecture using Metal Performance Shaders (MPS). Under our configuration, the M3 achieves consistently higher GPU utilization, oscillating around 60%, while consuming an order of magnitude less power than the GPUs on the HPC cluster. This divergence might result from the different architectures of the configurations. When training with small batch sizes, discrete cluster GPUs often become latency-bound. The computational workload per step is too small to hide the overhead of repeatedly transferring data across PCIe buses between the host CPU and the GPU. In contrast, Apple Silicon relies on a unified memory architecture where the CPU and GPU share a single physical memory pool. This eliminates transfer bottlenecks, allowing the M3 to maintain higher utilization. Combined with a mobile design that optimizes for performance per watt, the M3 processes small-batch workloads at a fraction of the energy cost of the SLURM cluster.

2.5 Conclusion

In conclusion, our findings highlight several practical implications regarding hardware selection, pipeline optimization, and energy efficiency in deep learning. First, **enterprise-grade hardware alone does not guarantee superior performance**. Without systematically inspecting and removing data transfer bottlenecks, **high-capacity GPUs remain severely underutilized, yielding diminished throughput relative to their cost and capability**.

Second, as anticipated, CPU throughput lagged significantly behind all specialized accelerators, confirming that even lightweight neural network architectures require dedicated parallel hardware support for time efficiency. In contrast, **Apple Silicon’s unified memory architecture bypasses the host-to-device latency inherent to traditional PCIe buses**. For small-batch training, this low-latency design allows consumer-grade hardware to match or surpass cluster GPUs, offering a viable and accessible alternative to reliance on high-performance computing (HPC) or cloud infrastructure.

Finally, **idle hardware imposes substantial energy and environmental costs**. Enterprise GPUs draw significant baseline power even during idle cycles. Operating cluster GPUs at low utilization leads to unnecessary financial expense and an unnecessarily high carbon footprint, **reinforcing the need for efficient pipeline management at scale**.

A GenAI Statement

During the project work GenAI tools have been used to support understanding of course content and development of practical implementations.

The tool Cursor (<https://cursor.com>) was used to ping pong ideas, summarize assignments into checklists and track progress. It was also used to support coding and unblocked progress when coding in an unfamiliar coding language. It was also used during report writing to generate scripts for processing the output data of experiments and produce figures for the report following clear requirement specifications from the caller. It was also used to parse results to find which metrics may be useful to enrich explanations of trends in the data that were identified by us, avoiding hours of manual parsing of quantitative results across files. Chatbot Claude (<https://claude.ai/>) was used for debugging, to generate code for plots, overleaf tables and figures and for spell and grammar check.

References

- [CR08] John Cieslewicz and Kenneth A. Ross. Data partitioning on chip multiprocessors. In *Proceedings of the 4th International Workshop on Data Management on New Hardware*, DaMoN '08, page 25–34, New York, NY, USA, 2008. Association for Computing Machinery.
- [DDS⁺09] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.
- [GG08] Kazushige Goto and Robert A. van de Geijn. Anatomy of high-performance matrix multiplication. *ACM Trans. Math. Softw.*, 34(3), May 2008.
- [RDK⁺22] William A Gaviria Rojas, Sudnya Diamos, Keertan Ranjan Kini, David Kanter, Vijay Janapa Reddi, and Cody Coleman. The dollar street dataset: Images representing the geographic and socioeconomic diversity of the world. In *Thirty-sixth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2022.
- [WXD⁺20] Bichen Wu, Chenfeng Xu, Xiaoliang Dai, Alvin Wan, Peizhao Zhang, Zhicheng Yan, Masayoshi Tomizuka, Joseph Gonzalez, Kurt Keutzer, and Peter Vajda. Visual transformers: Token-based image representation and processing for computer vision, 2020.